

Claude Skill

재사용 가능한 AI 자동화 레시피

한 번 만들면 누구나, 매번 동일한 품질로 · Claude가 적절한 시점에 자동 호출

claude.ai · Claude Code · Claude API 모든 환경에서 동일하게 동작

Skill이란?

Claude에게 특정 작업을 자동 반복시키는 재사용 가능한 지시서

What is a Skill?

Claude에게 특정 작업을 자동 반복시키는 재사용 가능한 지시서(레시피)

한 번 만들면 누구나 동일한 품질의 결과를 얻을 수 있다.

핵심 특징

- Model-invoked · Claude가 상황을 보고 자동 호출
- Progressive disclosure · 필요한 부분만 컨텍스트에 로드
- Composable · 여러 스킬을 조합해 복잡한 작업 수행

Skill 폴더 구조

공식 문서 기준

~/.claude/skills/

└─ my-skill/

└─ SKILL.md

← 필수 YAML + 지시문

└─ scripts/

← 선택 실행 스크립트

└─ run.py

└─ references/

← 선택 참고 문서

└─ guide.md

└─ assets/

← 선택 정적 파일 (이미지 등)

Skill 작동 방식

사용자는 평소처럼 요청할 뿐 · Claude가 알아서 적절한 스킬을 발동

Skill은 **Model-invoked**. 사용자가 명령어를 외울 필요 없이 **Claude가 description**을 보고 호출 여부를 판단한다.

01

SKILL.md 작성

작업 목적, 트리거 조건, 단계별 지시문, 출력 형식을 마크다운으로 정의

02

스킬 등록

Claude.ai 설정에서 업로드 · Claude Code는 ~/.claude/skills/ · API는 container 파라미터

03

Claude가 자동 발동

사용자 요청을 받으면 등록된 스킬들의 description을 스캔해서 관련 스킬을 컨텍스트에 로드

04

동일 품질로 결과 산출

스킬에 정의된 절차를 그대로 실행 · 매번 같은 입력은 같은 출력 · 팀 단위 표준화 가능

SKILL.md 작성법

YAML frontmatter로 메타데이터, 본문으로 지시문

description이 **Claude의 호출 판단 기준**. 어떤 상황에서 발동해야 하는지 **구체적인 키워드와 시나리오**로 기술한다.

```
---
name: data-cleaner
description:
  "CSV 파일의 결측치·이상치를 정리하는 스킬.
  '데이터 정리', 'CSV 클리닝' 키워드로 발동"
---

# Data Cleaner

## 입력
- CSV 파일 1개 (필수)

## 절차
1. 결측치 비율 점검 → 처리 전략 제안
2. 이상치 검출 → 사용자 승인 후 제거
3. 정제된 CSV + 보고서 출력
```

작성 팁

- description은 길고 구체적으로
- 트리거 키워드를 명시
- 절차는 번호 매긴 단계로
- 사용자 확인 시점을 명시
- 스크립트는 별도 파일로 분리
- 출력 형식·예시를 포함

팁 · description을 잘 쓰는 것이 스킬 품질의 절반. Claude는 description만 보고 호출 여부를 결정한다.

Skill 사용 환경

동일한 SKILL.md가 세 환경에서 그대로 동작

Anthropic은 **Skill을 오픈 표준으로 공개**했다. 한 번 만든 스킬을 **개인 · 팀 · 조직 단위**로 재사용한다.

Claude.ai

웹 · 모바일 채팅

Settings → Capabilities에서
스킬 업로드/관리

- GUI로 스킬 추가 · 삭제
- PPT · Excel · Word · PDF 사전 스킬 내장
- 엔터프라이즈는 조직 단위 공유 가능

Claude Code

터미널 · 개발 환경

~/.claude/skills/ 폴더에
파일로 저장

- 개인 스킬 · 프로젝트 스킬 분리
- git으로 팀 공유 가능
- /명령어로 직접 호출도 가능

Claude API

프로그램에서 호출

container 파라미터로
skill_id 지정

- Code execution tool과 함께 사용
- 사전 제작 + 커스텀 스킬 모두 지원
- Beta 헤더 필요 · 컨테이너에서 실행

포인트 · 스킬은 '프롬프트 엔지니어링의 결과물을 자산화'하는 방법. 한 번 만들면 영구 자산.

Case Study · csv-ml-pipeline

CSV 한 장으로 EDA → 시각화 → ML → Streamlit 앱까지 자동 생성

CSV 파일 하나를 던지면 **EDA 보고서 · 학습된 모델 · 추론 웹앱**이 자동 생성된다. 단순 일괄 처리가 아니라 **사용자 승인을 받으며 진행**하는 인터랙티브 설계.

발동 트리거 (description)

"CSV로 모델 만들어줘" · "이 데이터로 예측해줘"
"EDA 보고서" · "분류/회귀 모델" · "sklearn"
"streamlit 앱" · "추론 웹앱"

입력

- CSV 파일 1개 · 필수
- sklearn 모델명 · 선택 (미지정 시 자동 추천)

최종 산출물

output/ 폴더에 다음 파일들이 생성된다

<code>eda_report.html</code>	전문가 코멘트가 포함된 EDA 보고서
<code>wow_insight.html</code>	흥미로운 패턴 자동 탐색 (선택)
<code>model.pkl</code>	학습된 sklearn 모델
<code>evaluation.html</code>	ROC/PR · 혼동행렬 · 특성 중요도
<code>app.py + reqs.txt</code>	Streamlit 추론 웹앱

설계 철학 · 자동화하되 결정권은 사용자에게 · 타겟 컬럼/모델/처리 전략은 모두 사용자 승인 후 진행.

Case Study · 5 단계 인터랙티브 워크플로우

각 단계 사이에 사용자 승인 게이트가 있다

PHASE 1	EDA 실행	타겟 컬럼 확정 → run_pipeline.py --mode eda → eda_report.html 생성
PHASE 1-B	전문가 코멘트 자동 삽입	stats_summary.json 분석 → 5개 영역 코멘트 작성 → HTML에 삽입
 GATE	사용자 응답 대기	추가 시각화 / WOW VISUAL INSIGHT / ML 모델링 — 셋 중 하나 선택까지 정지
PHASE 2	추가 시각화 (반복 가능)	scatter · violin · box · pairplot · heatmap 등 9종 · 요청 1건 = 카드 1장
PHASE 3	ML 모델링	commentary.json 검토 → 불균형 5:1 초과 시 class_weight=balanced → 모델 학습
PHASE 4	완료 안내	model.pkl + evaluation_report + Streamlit 앱 · 실행 명령까지 안내
자동화 ≠ 자율 결정 타겟 · 모델 · 전략은 모두 사용자 확인을 거친다 환경 제약은 절대 규칙으로 matplotlib 한글 깨짐 → 영문만 사용을 강제 명시 번들 스크립트 신뢰 검증된 1000줄+ 스크립트는 절대 수정 금지로 못 박음		